

Valid Test Cases

These test cases successfully pass through the Scanner and Parser to produce a complete Abstract Syntax Tree (AST). They prove the structural logic of the front-end.

- **Variadic Arguments Test**
 - **Input:** `(+ 10 20 30 40)`
 - **Expected Behavior:** Successfully parses into a single addition operation with four numeric literal children.
- **Nested Expression Test**
 - **Input:** `(* (+ 1 2) (- 10 5))`
 - **Expected Behavior:** Successfully parses a nested structure where the outer multiplication operation takes two inner operations (addition and subtraction) as its arguments.
- **Variable Binding Test**
 - **Input:** `(let x 50) (* x 2)`
 - **Expected Behavior:** Successfully identifies the `let` keyword, binds the value 50 to the identifier `x`, and parses the subsequent multiplication operation.
- **Boolean Logic Test**
 - **Input:** `(and true (> 10 5))`
 - **Expected Behavior:** Successfully handles boolean literals (`true`) and logical operators alongside arithmetic comparisons.
- **Very Long Variadic Addition**
 - **Input:** `(+ 10 20 30 40 50 60 70 80 90 100)`
 - **Expected Behavior:** Successfully loops through and parses an extended list of ten arguments under a single operator, confirming the `while` loop logic for argument lists.
- **Complex Mixed Logic**
 - **Input:** `(and true (> (* 2 5) (+ 3 4)))`
 - **Expected Behavior:** Successfully parses deeply integrated logic and arithmetic, proving the recursive descent parser can handle mixed types accurately.

- **Division by Zero Test**
 - **Input:** `( / 10 0 )`
 - **Expected Behavior:** Success (Parsed). The code successfully builds a valid AST because the syntax is structurally correct. Literal division by zero is a semantic error, which falls outside the core front-end parsing scope of this workbench.

Invalid Test Cases

These test cases contain intentional errors to test the compiler's high-visibility diagnostic reporting and constraint enforcement.

- **Malformed Input Test (Missing Closing Parenthesis)**
 - **Input:** `( + 5 ( * 2 3 )`
 - **Expected Behavior:** The parser catches the missing delimiter and throws a Syntax Error: `[Line 1] Expect ')' after arguments.`
- **Invalid Character Test**
 - **Input:** `(let price $100)`
 - **Expected Behavior:** The scanner identifies the illegal \$ symbol and throws a Lexical Error: `Lexical Error at Line 1: Unexpected character: $`
- **Deeply Nested Expression (Parenthesis Mismatch)**
 - **Input:** `( + 1 ( + 2 ( + 3 ( + 4 ( + 5 ( + 6 ( + 7 ( + 8 9 ) ) ) ) ) ) ) ) )`
 - **Expected Behavior:** The parser verifies recursive stability by catching structural mismatches at extreme depths. Due to a missing closing parenthesis, it throws a Syntax Error: `[Line 1] Expected expression.`
- **Empty List Test**
 - **Input:** `( )`
 - **Expected Behavior:** The parser identifies that a list was opened but closed immediately without providing a required operator, throwing a Syntax Error: `[Line 1] Expected an operator after '(', but found ')'`.

- **Keyword Reservation Test**
 - **Input:** `(let and 10)`
 - **Expected Behavior:** The parser prevents the use of the reserved keyword `and` as an identifier and throws a Syntax Error: `[Line 1] Keyword Reservation Error: Cannot use 'and' as a variable name.`
- **Keyword Reservation Test 2**
 - **Input:** `(let let 10)`
 - **Expected Behavior:** The parser specifically prevents binding a variable to the `let` keyword itself and throws a Syntax Error: `[Line 1] Keyword Reservation Error: Cannot use 'let' as a variable name.`
- **Overflow Error Test**
 - **Input:** `(+ 999999999999 1)`
 - **Expected Behavior:** The scanner detects that the numeric literal exceeds the 32-bit integer limit (2,147,483,647) and throws a Lexical Error: `Lexical Error: Number '999999999999' is too large for 32-bit Integer.`